

Mode Progression Dynamics: How Work Moves Between Labor Modes in the Coordination Knowledge Substrate Pattern

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in "Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems" (Li, April 2026).

Author: Wenxin Li (Independent Researcher) **ORCID:** 0009-0004-8065-3235 **Date:** May 5, 2026 **License:** Creative Commons Attribution 4.0 International (CC BY 4.0)

Abstract

The Coordination Knowledge Substrate (CKS) labor allocation framework names three modes by which coordination work can be performed: direct human labor (Mode 1), LLM labor under orchestration rules (Mode 2), and stable-cell automation (Mode 3). The framework is named in the parent note A1.12 and integrated in A2.68; A2.69–A2.71 specialize the three modes individually; A2.72 specializes the three architectural properties making the modes coherent; A2.73 specializes the labor-vs-authority distinction. This note formalizes mode progression dynamics — the architectural properties governing how work moves between the three modes over time — as standalone architectural content. The dynamics are bidirectional: forward progressions (Mode 1 → 2, Mode 2 → 3) and reverse progressions (Mode 3 → 2, Mode 2 → 1) are equally architecturally supported. Forward progressions are human-driven through specific governance moments (rule authoring per A2.04 for Mode 1 → 2; stability assessment for Mode 2 → 3); reverse progressions are architecturally guaranteed by the temporal property of governance per A2.07, with no architectural justification precondition. Authority structure is preserved across all transitions per A2.73 — mode transitions involve labor allocation changes, not authority allocation changes. The note states the four operational components of the dynamics, distinguishes them from four adjacent operational patterns, names the failure modes that violate them, and provides an operational test.

1. Why mode progression dynamics need a standalone formalization

The labor allocation framework (per A1.12) names three modes that differ in who or what performs coordination work; the substrate content the modes produce is of the same architectural kind, governed by the same authority structure. The integrating-frame note A2.68 names mode progression dynamics at the integrating level. A2.69–A2.71 specialize each mode individually; A2.72 specializes the three architectural properties (single substrate, mode-independent authority, attributable writer); A2.73 specializes the labor-vs-authority distinction. None of these companion notes treats how work *moves* between modes as architectural content in its own right.

This note treats it as such. The architectural commitment is that all transitions — forward and reverse — are operationally feasible without architectural reconfiguration; that the three properties per A2.72 hold across each transition; and that authority structure per A2.73 is preserved through it. The motivating cases are deployments where work moves between modes over the deployment lifecycle: a deployment beginning in Mode 1 with humans performing coordination work directly, authoring rules over time as patterns become visible (Mode 1 → 2); rules reaching operational stability such that

per-execution review is no longer needed (Mode 2 → 3); a stable cell producing unexpected outputs that require return to per-execution attention (Mode 3 → 2); a rule found inadequate for new situations such that direct human labor resumes (Mode 2 → 1). Each progression requires specific architectural support that this note formalizes. The dynamics are load-bearing for several CKS commitments: A2.68's integrating frame becomes operationally meaningful only when the modes connect through transitions; A2.72's three properties must hold *across* transitions, not only within each mode; A2.07's temporal property of governance is what guarantees reverse progressions; and A1.11's non-specialist governance commitment depends on mode flexibility persisting across the deployment lifecycle.

2. Mode 1 → Mode 2 progression

Mode 1 → Mode 2 progression occurs when humans author orchestration rules per A2.04 for work that has been performed in Mode 1. The progression has four operational components.

(a) Pattern recognition. Humans performing Mode 1 labor observe patterns in the work — repeated decisions, consistent rationale, applicable structures. Pattern recognition is itself a Mode 1 governance activity exercised during direct labor.

(b) Rule authoring. Humans author orchestration rules per A2.04 capturing the patterns. The rules become substrate content (per A2.46, Category 4 source-of-truth: substrate authoritative for "what rules apply") and are subject to the human-governed authority structure per A1.01.

(c) Cell instantiation. Cells are authored per A2.66's authorship roles to operate under the new rules. The cells satisfy the five mediator properties per A2.19–A2.23: the LLM reads from substrate as primary source of state, writes under rules, holds no substrate-relevant state outside the substrate, exercises no authority over substrate content, and produces outputs recorded in substrate with attribution.

(d) Mode transition. Subsequent work in the rule's scope is performed by LLMs in cells under the rule (Mode 2) rather than by humans directly (Mode 1). Authority over the resulting substrate content is preserved per A2.73 — humans who held authority over Mode 1 content within their authority scope per A2.47 retain authority when work shifts to Mode 2.

The progression is human-driven (rule authoring per A2.04 is a human governance moment); the timing is a deployment choice; the architectural commitment is that the progression is operationally feasible without changing authority structure.

3. Mode 2 → Mode 3 progression

Mode 2 → Mode 3 progression occurs when orchestration rules reach operational stability per A2.71's stable-cell automation. The progression has four operational components.

(a) Stability assessment. Humans assess that the orchestration rule has produced rule-conformant outputs across many executions, with the rule being refined and validated to operational stability. Stability assessment is a human governance moment; the criteria are deployment choices.

(b) Attention reduction. Humans reduce per-execution review attention; review moves to coarser granularity (per-rule rather than per-execution).

(c) Operational adjustment. The deployment's operational processes adjust — fewer per-execution review interfaces, coarser monitoring, longer review intervals.

(d) Mode transition. The cell continues to operate under the same orchestration rule, but with the operational attention pattern of Mode 3 rather than Mode 2. The five mediator properties per A2.19–A2.23 continue to hold; the temporal property of governance per A2.07 ensures humans can return attention at any moment. Authority over the resulting substrate content is preserved per A2.73.

The progression is human-driven (humans assess stability and adjust attention); the criteria for stability are deployment choices; the architectural commitment is that the progression is operationally feasible while preserving the at-any-time governance commitment per A2.07.

4. Reverse progressions: Mode 3 → Mode 2 and Mode 2 → Mode 1

Reverse progressions occur when operational conditions change such that finer-granularity human attention is required. They are architecturally guaranteed by A2.07's temporal property — humans may exercise the three rights per A2.01–A2.03 at any moment, including override per A2.03 with no architectural justification precondition.

Mode 3 → Mode 2 operates as follows.

(a) Trigger event. A stable cell produces an unexpected output; operational conditions change; auditing reveals concerns; or humans choose to return to per-execution attention for specific reasons. No architectural justification is required (per A2.03).

(b) Attention restoration. Humans restore per-execution review attention. The cell continues to operate under the same orchestration rule, but with Mode 2's per-execution attention pattern.

(c) Operational adjustment. The deployment's operational processes adjust to support per-execution review.

(d) Authority continuity. Authority over the resulting substrate content is preserved per A2.73; the same humans who held authority during Mode 3 hold authority during Mode 2.

Mode 2 → Mode 1 operates as follows.

(a) Inadequacy recognition. The orchestration rule is found inadequate for new situations — its scope does not cover the situation, or it produces inappropriate outputs.

(b) Direct labor resumption. Humans take direct labor for the inadequate situations — humans read substrate per A2.01, modify per A2.02, or override per A2.03 directly.

(c) Optional rule update. Humans may author a revised rule per A2.04 to handle the situation in Mode 2 going forward, or continue Mode 1 labor for the situation indefinitely.

(d) Authority continuity. Authority over substrate content is preserved per A2.73 across the transition.

The reverse progressions are architecturally distinguished from forward progressions by their triggers and operational mechanism. Forward progressions require active governance (someone authors rules; someone assesses stability); reverse progressions require only the standing temporal property per

A2.07. The mechanisms differ; the feasibility commitment is symmetric.

5. Authority preservation across all transitions

Authority preservation across all mode transitions is a load-bearing architectural commitment. Per A2.73's labor-vs-authority distinction, authority is mode-invariant; mode transitions do not change authority structure. Four properties together specify what this preservation requires architecturally.

(a) Same-authority continuity. Humans who held authority over substrate content within their authority scope per A2.47 retain that authority when work shifts modes. Authority is determined by the substrate scope, not by the labor mode that produced the content.

(b) No authority delegation through labor delegation. Mode transitions involve labor allocation changes (different actors performing work) but not authority allocation changes. The architectural right to govern remains with humans per A1.01; the labor-vs-authority distinction per A2.73 holds across transitions.

(c) Provenance preservation. Substrate content carries provenance per A2.40 indicating which mode produced it (writer attribution per A2.37 distinguishes Mode 1, Mode 2, and Mode 3 attribution). Mode transitions do not erase or modify the provenance of previously produced content; the change is in subsequent work's mode, not in past work's record.

(d) At-any-time governance preservation. The temporal property per A2.07 ensures humans can exercise the three rights per A2.01–A2.03 at any moment, including during and immediately after mode transitions. There is no architectural delay or precondition for governance exercise during transitions.

Authority preservation is what makes mode progression operationally feasible without governance discontinuities. Without it, transitions would introduce gaps where labor reallocation outpaces authority continuity, breaking the framework's adaptive character.

6. What the dynamics do not claim, and four adjacent patterns they are not

What the dynamics do not claim. They do not claim that all progressions are equally common — forward progressions are operationally common as deployments mature; reverse progressions occur less frequently in stable deployments, and the architectural commitment is that all are *feasible*, not that they occur equally often. They do not specify timing or criteria for any progression — when to author rules, when to assess stability, when to restore attention are deployment choices. They do not foreclose simultaneous mixed-mode operation — a deployment may have work in all three modes simultaneously per A2.68's integrating frame; mode progression operates per work category, not at the deployment level. They do not require symmetric reversibility of mechanism — forward progressions involve specific governance moments (rule authoring per A2.04, stability assessment per A2.71); reverse progressions occur through the temporal property per A2.07; the mechanisms differ, the feasibility commitment is symmetric. They do not claim mode progressions produce specific outcomes; the architectural commitment is to the operational components, not to specific tools, workflows, or outcomes.

Four adjacent patterns the dynamics are not.

Not workflow automation. Workflow automation replaces manual workflows with automated systems; it operates outside the labor allocation framework. Mode progression dynamics operate within the framework's three modes. Workflow automation that does not preserve the modes' architectural distinctions (e.g., direct replacement of human labor with autonomous AI without rule-governance) is not Mode 1 → 2 progression; it is architectural drift away from the framework.

Not system migration. System migration moves deployments from one infrastructure to another; mode progression dynamics operate within a single deployment's labor allocation, not across deployments. A migration that preserves substrate content per A1.05 may also involve mode allocations on the new host, but the migration itself is not mode progression.

Not capability maturation. Capability maturation describes capabilities improving over time; mode progression dynamics specifically address how labor allocation evolves. Capability maturation may accompany mode progression but is not architecturally specified by it — a deployment's Mode 1 capabilities may improve while remaining in Mode 1.

Not organizational change. Organizational change describes how humans organize around work; mode progression dynamics operate at the architectural level. Organizational change may accompany mode progression (e.g., teams reorganizing around rule authoring after a Mode 1 → 2 progression) but is not architecturally specified. The architectural commitment is to mode flexibility; organizational consequences are deployment-specific.

7. Failure modes

Each failure mode names a way an implementation can fail mode progression dynamics by treating progressions as one-way or by compromising authority across transitions.

(a) One-way Mode 1 → Mode 2. The implementation supports Mode 1 → 2 but not Mode 2 → 1; once work moves to Mode 2 cells, it cannot return to direct human labor for specific situations.

(b) One-way Mode 2 → Mode 3. The implementation supports Mode 2 → 3 but not Mode 3 → 2; once cells reach operational stability and move to Mode 3, they cannot return to per-execution review. The temporal property of governance per A2.07 fails for Mode 3 cells.

(c) Authority loss across transitions. Authority structure changes across transitions — humans who held authority in Mode 1 lose authority when work transitions to Mode 2, with new authority structures (perhaps centralized in specialists) replacing the original authority. Authority preservation per §5(a) fails.

(d) Provenance erasure across transitions. Mode transitions erase or modify the provenance of previously produced content; transitions become operationally indistinguishable in the substrate record. Retraceability per A1.07 is compromised.

(e) Mode transitions as architectural reconfiguration. Transitions require architectural reconfiguration — new substrates, new cell architectures, new authority structures — making them operationally expensive and architecturally disruptive. The commitment to mode progression as operationally feasible fails.

(f) Justification-required reverse progressions. Humans must provide architectural justifications for reverse progressions (e.g., rule-update proposals as preconditions for Mode 3 → 2 reversal). The no-justification-as-precondition commitment per A2.03 fails for reverse progressions specifically.

(g) Mode 1 disablement post-transition. After Mode 1 → 2 progression, humans can no longer exercise direct labor for the work category; reverse progression Mode 2 → 1 is not architecturally supported.

(h) Implicit mode progression. The implementation automatically promotes Mode 2 cells to Mode 3 based on operational metrics (e.g., rule-conformance rates) without human authorization. The commitment to mode progression being human-driven (with rule authoring per A2.04 and stability assessment per A2.71 as human governance moments) fails. This failure mode is the most likely to be encoded in commercial AI architectures presenting themselves as human-governed: progression is rendered an emergent system property rather than an exercise of human authority.

(i) Mode-incompatible deployment. Deployments must commit to a single mode at deployment time — Mode 1-only, Mode 2-only, or Mode 3-only — with no support for mode progression over the lifecycle. The mixed-mode commitment per A2.68 fails alongside the progression dynamics.

(j) Authority-shifting reverse progressions. Reverse progressions are supported, but with authority shifting across the transition (e.g., authority returning to humans who were not the original authority-holders). Authority continuity per §5(a) fails.

8. Operational test

A system supports mode progression dynamics if and only if all of the following are true at all times during the substrate's existence.

1. Mode 1 → Mode 2 progression is operationally feasible per §2 — humans can author orchestration rules per A2.04 and instantiate cells under those rules.
2. Mode 2 → Mode 3 progression is operationally feasible per §3 — humans can reduce per-execution attention as rules reach operational stability.
3. Reverse progressions Mode 3 → Mode 2 and Mode 2 → Mode 1 are operationally feasible per §4 — humans can restore per-execution attention or resume direct labor at any moment per A2.07, with no architectural justification precondition per A2.03.
4. Authority is preserved across all transitions per §5 — the four authority-preservation properties hold for all progression types.
5. Mode progressions are work-scoped, not deployment-scoped; a deployment may have work in different modes simultaneously per A2.68.
6. Mode progressions are human-driven for forward progressions (rule authoring per A2.04 for Mode 1 → 2; stability assessment per A2.71 for Mode 2 → 3) and architecturally guaranteed for reverse progressions (temporal property per A2.07).

A system that fails any of (1)–(6) does not support mode progression dynamics in the architectural sense, even if it operationally supports some mode transitions.

Conclusion

Mode progression dynamics are what make the CKS labor allocation framework operationally adaptive. Without them, the framework would be static at deployment time — labor allocation fixed at the moment of deployment, with no architectural support for evolution as patterns become visible, rules stabilize, conditions change, or operational concerns emerge. With them, the framework supports the deployment lifecycles the source paper's §2.3 develops: deployments beginning entirely in Mode 1, authoring rules as patterns emerge, letting work under stable rules move to Mode 3, and returning work to finer-granularity attention when conditions warrant — all without changing the substrate's governance structure.

Implementations under pressure to simplify deployment lifecycle drift consistently toward one-way progression patterns or fixed-mode deployments. The drift is steady because one-way progression is operationally simpler and fixed-mode deployment is commercially appealing (vendors can specialize in specific labor modes). Implementations that drift away from bidirectional progression produce systems where the labor allocation framework is operationally rigid: governance fails when conditions warrant return to direct labor, the at-any-time commitment fails for cells that cannot return to per-execution attention, and the framework's adaptive character collapses into a static allocation specified at deployment time.

This note completes the labor-allocation decomposition (A2.68–A2.74). The integrating frame established the structure (A2.68); the three modes were specialized individually (A2.69–A2.71); the three architectural properties were specialized as a unified specification (A2.72); the labor-vs-authority distinction was specialized (A2.73); this note specializes how work moves between the modes while preserving the architectural commitments. Subsequent work that adopts the labor allocation framework should use "mode progression dynamics" in the bidirectional sense formalized here.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Mode Progression Dynamics: How Work Moves Between Labor Modes in the Coordination Knowledge Substrate Pattern*. Derivation Note. May 5, 2026. ORCID: 0009-0004-8065-3235.